



Complete Beginner's Guide

Step-by-step instructions with real code — no experience needed

#	Section	What you will do
1	Setup & Installation	Install Python, PostgreSQL, Django on your computer
2	Database Setup	Create your PostgreSQL database and tables
3	Fetch API Data	Write a Python script to pull data from your API
4	Save to PostgreSQL	Store the live API data into your database
5	Build Django App	Create the Django web application
6	Display on Website	Show live data on a webpage
7	Auto-Refresh (Live)	Schedule automatic data updates
8	Final Checklist	Verify everything works end-to-end

■ Before We Start — Understanding the Big Picture

What are we building?

You have an **API** (a web address that returns data, like weather or prices). We will build a system that: **(1)** fetches that data automatically, **(2)** saves it to a **PostgreSQL database** (like a super-powered spreadsheet), and **(3)** shows it on a **website** built with **Django** (a Python web framework).

Term	Simple Explanation	Real-world analogy
API	A web address that gives you data in JSON format	Like a vending machine — you press a button, it gives you a snack
PostgreSQL	A database program that stores your data permanently	Like a filing cabinet that never loses files
Django	A Python tool that builds websites quickly	Like a pre-built house kit — structure already done for you
Python	The programming language we use to write all our code	The language you speak to give instructions to the computer
JSON	The format APIs return data in (looks like a dictionary)	<code>{ "name": "Ahmed", "age": 30 }</code>
Virtual Environment	An isolated folder for your project's tools	Like a separate toolbox so tools don't mix with other projects

■ **TIP:** Read this section carefully before writing any code. Understanding WHY you are doing each step makes the whole guide much easier to follow.

■ ■ Section 1 — Install Everything You Need

1A — Install Python

Python is the language all our code is written in. Follow these steps:

- Go to <https://www.python.org/downloads/> in your browser
- Click the big yellow **'Download Python 3.x.x'** button
- Run the installer. **IMPORTANT:** tick **'Add Python to PATH'** before clicking Install
- Open your terminal (Command Prompt on Windows / Terminal on Mac) and type:

```
python --version
# You should see something like: Python 3.11.4
```

■ **TIP:** On Windows press the Windows key, type 'cmd', and press Enter to open the terminal.

1B — Install PostgreSQL

- Go to <https://www.postgresql.org/download/>
- Choose your operating system (Windows / Mac / Linux) and download the installer
- Run the installer. When it asks for a **password**, remember it — you will need it later
- Keep the default port **5432** and click through to finish
- After installation, open **pgAdmin** (installed automatically) to verify it works

■ ■ **NOTE:** Write down your PostgreSQL password somewhere safe. You cannot recover it easily.

1C — Set Up Your Project Folder

Open your terminal and run these commands one by one:

```
# Create a project folder
mkdir my_api_project
cd my_api_project

# Create a virtual environment (isolated workspace)
python -m venv venv

# Activate the virtual environment
# On Windows:
venv\Scripts\activate
# On Mac/Linux:
source venv/bin/activate

# You will see (venv) appear at the start of your terminal line
# This means it worked!
```

■ **TIP:** Always activate your virtual environment before running any Python commands in this project. You will know it is active because you see (venv) at the start of your terminal.

1D — Install Python Libraries

With your virtual environment active, install all needed libraries:

```
pip install django psycopg2-binary requests python-dotenv

# What each library does:
# django          - builds your website
# psycopg2-binary - connects Python to PostgreSQL
# requests        - fetches data from your API
# python-dotenv   - keeps your passwords safe in a .env file
```

■ ■ Section 2 — Create Your PostgreSQL Database

2A — Create the Database

Open **pgAdmin** (the PostgreSQL graphical tool). In the left panel, right-click **Databases** → **Create** → **Database**. Name it **api_data_db** and click Save.

Alternatively, use the terminal:

```
# Open the PostgreSQL command line tool
psql -U postgres

# Inside psql, create the database
CREATE DATABASE api_data_db;

# Create a dedicated user (safer than using 'postgres')
CREATE USER api_user WITH PASSWORD 'your_strong_password';

# Give the user access
GRANT ALL PRIVILEGES ON DATABASE api_data_db TO api_user;

# Exit psql
\q
```

■ ■ **NOTE:** Replace 'your_strong_password' with a real password of your choice.

2B — Create a .env File (Keeps Secrets Safe)

In your project folder, create a file called **.env** (exactly that name, with the dot). This file stores sensitive information so it is never accidentally shared:

```
# File: .env (create this in your my_api_project folder)

DB_NAME=api_data_db
DB_USER=api_user
DB_PASSWORD=your_strong_password
DB_HOST=localhost
DB_PORT=5432
API_URL=https://your-api-url-here.com/endpoint
API_KEY=your_api_key_if_needed
```

■ **TIP:** Add '.env' to your .gitignore file if you ever use Git, so your passwords are never uploaded.

■ Section 3 — Create Your Django Project

3A — Start a New Django Project

```
# Make sure you are inside my_api_project and venv is active
django-admin startproject mysite .

# The dot '.' at the end is important!
# Create a Django 'app' (a module inside the project)
python manage.py startapp livedata
```

Your folder structure will now look like this:

```
my_api_project/
├── .env                ← your secrets file
├── manage.py           ← Django's control script
├── mysite/
│   ├── settings.py     ← project settings
│   ├── urls.py         ← website URL routes
│   └── wsgi.py
├── livedata/           ← your app
│   ├── models.py       ← database table definitions
│   ├── views.py        ← what each page does
│   └── templates/      ← HTML files for pages
└── venv/               ← virtual environment (don't touch)
```

3B — Configure settings.py

Open **mysite/settings.py** in any text editor and make these changes:

```
# At the very TOP of settings.py, add:
import os
from dotenv import load_dotenv
load_dotenv() # Loads your .env file

# Find INSTALLED_APPS and add 'livedata':
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'livedata', # <-- ADD THIS LINE
]

# Find DATABASES and replace it with:
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.getenv('DB_NAME'),
        'USER': os.getenv('DB_USER'),
        'PASSWORD': os.getenv('DB_PASSWORD'),
```

```
    'HOST':      os.getenv('DB_HOST', 'localhost'),  
    'PORT':      os.getenv('DB_PORT', '5432'),  
  }  
}
```

■ Section 4 — Define Your Database Table (Models)

4A — What is a Model?

A **model** in Django is a Python class that defines what your database table looks like. Each attribute becomes a column. Below is a generic example — adapt the fields to match what your API actually returns:

```
# File: livedata/models.py

from django.db import models

class ApiRecord(models.Model):
    """
    Stores one row of data fetched from the API.
    Adapt the fields below to match your API's response.
    """
    # Common field types:
    title = models.CharField(max_length=255) # short text
    description = models.TextField(blank=True) # long text
    value = models.FloatField(null=True) # number with decimals
    count = models.IntegerField(default=0) # whole number
    is_active = models.BooleanField(default=True) # True/False
    created_at = models.DateTimeField(auto_now_add=True) # set on insert
    updated_at = models.DateTimeField(auto_now=True) # updated every save

    class Meta:
        ordering = ['-created_at'] # newest first

    def __str__(self):
        return self.title
```

■ **TIP:** Look at what your API returns (open the API URL in a browser). Each key in the JSON response should become a field in your model.

4B — Apply the Model to Create the Database Table

```
# In your terminal (with venv active):

# Step 1: Create migration files (tells Django what changed)
python manage.py makemigrations

# Step 2: Apply migrations (actually creates the table in PostgreSQL)
python manage.py migrate

# You should see: 'Applying livedata.0001_initial... OK'
```

■ **NOTE:** Run 'makemigrations' and 'migrate' every time you change models.py.

■ Section 5 — Fetch API Data & Save to PostgreSQL

5A — Write the Data Fetcher Script

Create a file `livedata/fetch_data.py`. This script fetches data from your API and saves it to PostgreSQL:

```
# File: livedata/fetch_data.py

import os, django, requests
from dotenv import load_dotenv

load_dotenv()

# Tell Python where the Django project settings are
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'mysite.settings')
django.setup()

from livedata.models import ApiRecord

def fetch_and_save():
    api_url = os.getenv('API_URL')
    api_key = os.getenv('API_KEY', '') # optional

    headers = {}
    if api_key:
        headers['Authorization'] = f'Bearer {api_key}'

    try:
        response = requests.get(api_url, headers=headers, timeout=10)
        response.raise_for_status() # raises error if status != 200
        data = response.json()

        # data is usually a list or has a key like data['results']
        # Adjust based on YOUR API response structure:
        records = data if isinstance(data, list) else data.get('results', [data])

        for item in records:
            ApiRecord.objects.update_or_create(
                title=item.get('title', 'No Title'),
                defaults={
                    'description': item.get('description', ''),
                    'value': item.get('value'),
                    'count': item.get('count', 0),
                    'is_active': item.get('is_active', True),
                }
            )
        print(f'Saved {len(records)} records successfully.')

    except requests.RequestException as e:
        print(f'Error fetching API: {e}')

if __name__ == '__main__':
    fetch_and_save()
```

5B — Run it for the first time

```
python livedata/fetch_data.py  
# Expected output: 'Saved X records successfully.'
```

■ **TIP:** 'update_or_create' is smart — it updates a record if it already exists, or creates a new one. This prevents duplicates.

■ ■ Section 6 — Display Live Data on Your Website

6A — Write the View (livedata/views.py)

A view is a Python function that decides what to show on each page:

```
# File: livedata/views.py

from django.shortcuts import render
from .models import ApiRecord

def index(request):
    records = ApiRecord.objects.all() # get all data from DB
    return render(request, 'livedata/index.html', {'records': records})

def detail(request, pk):
    from django.shortcuts import get_object_or_404
    record = get_object_or_404(ApiRecord, pk=pk)
    return render(request, 'livedata/detail.html', {'record': record})
```

6B — Set Up URLs

First, create `livedata/urls.py` (new file):

```
# File: livedata/urls.py (create this new file)

from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('record/', views.detail, name='detail'),
]
```

Then edit `mysite/urls.py` to include your app's URLs:

```
# File: mysite/urls.py

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('livedata.urls')), # <-- ADD THIS LINE
]
```

6C — Create the HTML Templates

Create this folder structure:

```
livedata/
  ■ ■ ■ templates/
    ■ ■ ■ livedata/
```

■■■ base.html	← shared layout
■■■ index.html	← list of all records
■■■ detail.html	← single record detail

Create `livedata/templates/livedata/base.html`:

```
<!-- File: base.html -->
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='UTF-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1'>
  <title>Live API Dashboard</title>
  <link rel='stylesheet' href=
    'https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css'>
</head>
<body class='bg-light'>
  <nav class='navbar navbar-dark bg-dark px-4'>
    <span class='navbar-brand'>■ Live API Dashboard</span>
  </nav>
  <div class='container mt-4'>
    {% block content %}{% endblock %}
  </div>
</body>
</html>
```

Create `livedata/templates/livedata/index.html`:

```
<!-- File: index.html -->
{% extends 'livedata/base.html' %}
{% block content %}
<h2>All Records ({{ records|length }} total)</h2>
<a href='.' class='btn btn-primary mb-3'>■ Refresh</a>
<div class='table-responsive'>
  <table class='table table-striped table-hover'>
    <thead class='table-dark'>
      <tr>
        <th>#</th>
        <th>Title</th>
        <th>Value</th>
        <th>Active</th>
        <th>Last Updated</th>
        <th>Detail</th>
      </tr>
    </thead>
    <tbody>
      {% for r in records %}
      <tr>
        <td>{{ forloop.counter }}</td>
        <td>{{ r.title }}</td>
        <td>{{ r.value }}</td>
        <td>{% if r.is_active %}■{% else %}■{% endif %}</td>
        <td>{{ r.updated_at|date:'d M Y H:i' }}</td>
        <td><a href='/record/{{ r.pk }}/'>View</a></td>
      </tr>
      {% empty %}
      <tr><td colspan='6'>No data yet. Run the fetch script!</td></tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endblock %}
```

■ ■ Section 7 — Turn Fetcher into a Django Command

Converting the fetch script into a Django management command lets you run it with **python manage.py fetch_api_data** — cleaner and easier to schedule.

```
# Create these folders:
livedata/management/__init__.py          (empty file)
livedata/management/commands/__init__.py (empty file)
livedata/management/commands/fetch_api_data.py

# File: fetch_api_data.py
import os, requests
from django.core.management.base import BaseCommand
from livedata.models import ApiRecord
from dotenv import load_dotenv

load_dotenv()

class Command(BaseCommand):
    help = 'Fetch data from API and save to PostgreSQL'

    def handle(self, *args, **options):
        api_url = os.getenv('API_URL')
        api_key = os.getenv('API_KEY', '')
        headers = {'Authorization': f'Bearer {api_key}'} if api_key else {}

        try:
            r = requests.get(api_url, headers=headers, timeout=10)
            r.raise_for_status()
            data = r.json()
            records = data if isinstance(data, list) else data.get('results', [data])
            count = 0
            for item in records:
                ApiRecord.objects.update_or_create(
                    title=item.get('title', 'No Title'),
                    defaults={
                        'description': item.get('description', ''),
                        'value':      item.get('value'),
                        'count':      item.get('count', 0),
                        'is_active':  item.get('is_active', True),
                    }
                )
                count += 1
            self.stdout.write(self.style.SUCCESS(f'Saved {count} records.'))
        except Exception as e:
            self.stderr.write(f'Error: {e}')

# Run the command:
python manage.py fetch_api_data
```

■ Section 8 — Auto-Refresh Data (Live Updates)

8A — Schedule with Cron (Mac / Linux)

```
# Open crontab editor:
crontab -e

# Add this line to run every 5 minutes:
*/5 * * * * /path/to/venv/bin/python /path/to/my_api_project/manage.py fetch_api_data

# Example with real paths:
*/5 * * * * /home/user/my_api_project/venv/bin/python /home/user/my_api_project/manage.py fetch_api_data >> /home/user/fetch.log 2>&1
```

8B — Schedule on Windows (Task Scheduler)

- Press Windows key → search 'Task Scheduler' → Open it
- Click 'Create Basic Task' in the right panel
- Name it 'Fetch API Data', click Next
- Choose 'Daily' or 'When the computer starts', click Next
- Choose 'Start a Program', click Next
- Program: **C:\path\to\my_api_project\venv\Scripts\python.exe**
- Arguments: **manage.py fetch_api_data**
- Start in: **C:\path\to\my_api_project**
- Click Finish

8C — Auto-Refresh the Webpage (Optional)

Add this tag inside the <head> of base.html to refresh every 60 seconds:

```
<!-- In base.html inside <head> -->
<meta http-equiv='refresh' content='60'>
```

■ Section 9 — Run & Test Your Website

9A — Start the Django Development Server

```
# With venv active, in your project folder:
python manage.py runserver

# You will see:
# Starting development server at http://127.0.0.1:8000/
# Quit the server with CTRL-BREAK (Windows) or Ctrl+C (Mac/Linux)
```

Open your browser and go to **http://127.0.0.1:8000/**. You should see your data!

9B — Set Up the Admin Panel (Bonus!)

Django has a built-in admin panel to view and edit data:

```
# Create an admin superuser:
python manage.py createsuperuser
# Enter a username, email, and password when prompted

# Register your model in livedata/admin.py:
from django.contrib import admin
from .models import ApiRecord

admin.site.register(ApiRecord)

# Access admin at: http://127.0.0.1:8000/admin/
```

■ Section 10 — Common Problems & Fixes

Error Message	Likely Cause	Fix
OperationalError: connection refused	PostgreSQL is not running	Open pgAdmin or run: <code>sudo service postgresql start</code>
ModuleNotFoundError: No module named 'django'	Virtual environment not active	Run: <code>venv\Scripts\activate</code> (Windows) or <code>source venv/bin/activate</code> (Mac/Linux)
django.db.utils.OperationalError: no such table: migration	Migrations not applied	Run: <code>python manage.py migrate</code>
ConnectionRefusedError (API)	Wrong API URL or no internet	Double-check <code>API_URL</code> in <code>.env</code> file
TemplateDoesNotExist	Template file in wrong folder	Check folder path: <code>livedata/templates/livedata/index.html</code>
ImproperlyConfigured: DATABASES	Wrong DB settings in <code>settings.py</code>	Check <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> in <code>.env</code>
Page not found (404)	URL not configured	Check <code>livedata/urls.py</code> and <code>mysite/urls.py</code>
JSONDecodeError	API returned non-JSON response	Print <code>response.text</code> to see what the API actually returns

■ Final Checklist — Did Everything Work?

■	Task	Verification
■	Python installed and working	python --version shows 3.x
■	PostgreSQL installed and running	pgAdmin opens without error
■	Virtual environment created and active	(venv) visible in terminal
■	All libraries installed	pip list shows django, psycopg2, requests
■	.env file created with correct credentials	File exists in project root
■	Django project created	manage.py file exists
■	PostgreSQL database created	api_data_db exists in pgAdmin
■	models.py written and migrated	python manage.py migrate ran OK
■	fetch_api_data command works	Prints 'Saved X records'
■	views.py and urls.py configured	No import errors
■	HTML templates created	3 files in livedata/templates/livedata/
■	Website shows data in browser	http://127.0.0.1:8000/ loads data
■	Auto-refresh scheduled	Cron or Task Scheduler set up

■ Congratulations — You Did It!

You now have a fully working system that automatically fetches live data from your API, stores it in a professional PostgreSQL database, and displays it on a beautiful website built with Django. Every time the scheduled task runs, your website will show updated data automatically. Well done — this is real software engineering!